

Full-Order State Observers

Estimating the state you cannot measure.

Ogata, *Modern Control Engineering*, Ch. 10.

In this tutorial you will:

- build a Luenberger observer and design its gain G by duality,
- simulate the estimate converging to the true state, and
- see the speed/noise tradeoff in choosing the observer poles.

Step through with **Ctrl+Enter**, or render a report with **publish**.

When not all states are directly measurable, a **Luenberger observer** reconstructs an estimate $\hat{x}(t)$ from the measured input/output:

$$\dot{\hat{x}} = A\hat{x} + Bu + G(y - C\hat{x})$$

Defining the estimation error $e = x - \hat{x}$:

$$\dot{e} = (A - GC)e$$

so the error decays to zero (at a rate set by the eigenvalues of $A - GC$) independent of the input, provided (A, C) is observable and G is chosen to place those eigenvalues in the left half-plane.

Contents

- Plant
- Observer Gain Design by Duality
- Observer Simulation
- Choosing Observer Pole Speed
- What Changes with Observer Pole Speed: Error Convergence
- Try it yourself

Plant

```
A = [0 1 0; 0 0 1; 0 -2 -3];  
B = [0;0;1];  
C = [1 0 0];  
D = 0;  
fprintf('Observability rank = %d (n=%d)\n', rank(observ(A,C)), size(A,1))
```

```
Observability rank = 3 (n=3)
```

Observer Gain Design by Duality

Because $(A - GC)$ and $(A^T - C^T G^T)$ have the same eigenvalues, G is found by applying ‘acker’/‘place’ to the **dual** pair (A^T, C^T) and transposing the result:

```
observer_poles = [-10, -10.5, -11]; % faster than the controller poles  
G = acker(A',C',observer_poles)';  
fprintf('Observer gain G = \n'); disp(G)  
fprintf('Eigenvalues of A-GC: '); disp(eig(A-G*C))
```

```

Observer gain G =
    28.5000
   243.0000
   369.0000

Eigenvalues of A-GC:   -11.0000   -10.5000   -10.0000

```

Observer Simulation

Simulate the true plant alongside the observer driven by the same input and the plant's measured output, starting from different initial states to show error convergence.

```

sys_plant = ss(A,B,C,D);
t = 0:0.01:3;
u = sin(2*t);
x0_true = [1;0.5;-0.5];
x0_hat = [0;0;0];

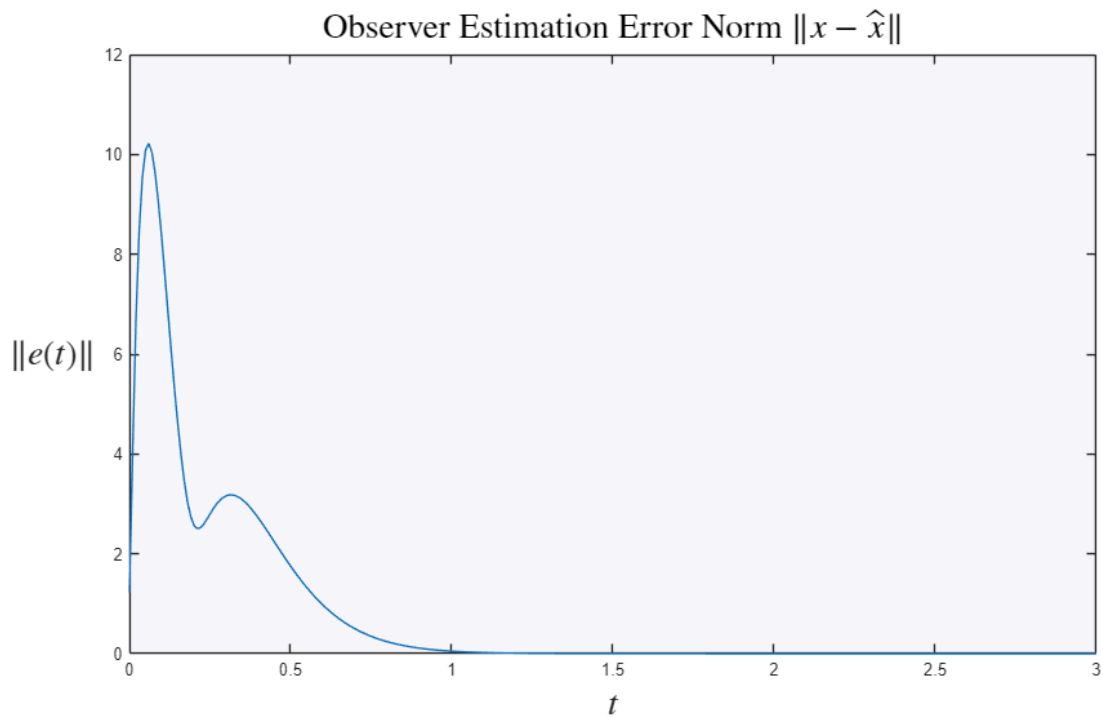
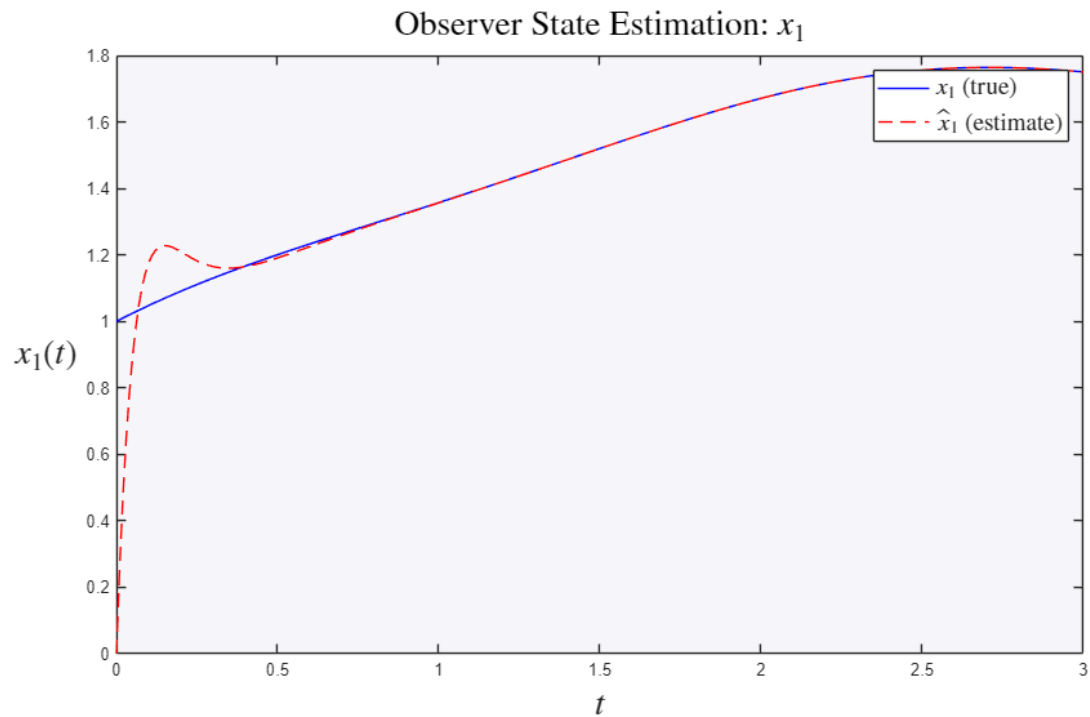
[y_true,~,x_true] = lsim(sys_plant,u,t,x0_true);

% Observer dynamics: augmented state [x_hat] driven by u and y(measured)
A_obs = A - G*C;
B_obs = [B G];
C_obs = eye(3);
D_obs = zeros(3,2);
sys_obs = ss(A_obs,B_obs,C_obs,zeros(3,2));
u_aug = [u; y_true'];
[x_hat,~] = lsim(sys_obs,u_aug,t,x0_hat);

figure
plot(t,x_true(:,1),'b',t,x_hat(:,1),'r--')
legend('$x_1$ (true)', '$\hat{x}_1$ (estimate)', 'Interpreter','latex','FontSize',14)
title('Observer State Estimation: $x_1$', 'Interpreter','latex','FontSize',20)
ylabel('$x_1(t)$', 'Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$', 'Interpreter','latex','FontSize',20)

figure
plot(t, vecnorm(x_true-x_hat,2,2))
title('Observer Estimation Error Norm $\|x-\hat{x}\|$', 'Interpreter','latex','FontSize',20)
ylabel('$\|e(t)\|$', 'Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$', 'Interpreter','latex','FontSize',20)

```



Choosing Observer Pole Speed

Observer poles are conventionally placed 2-10x faster (more negative) than the controller poles, so estimation error decays quickly relative to the controlled response – but excessive speed amplifies sensor noise sensitivity through G (a practical tradeoff, since G grows with pole speed).

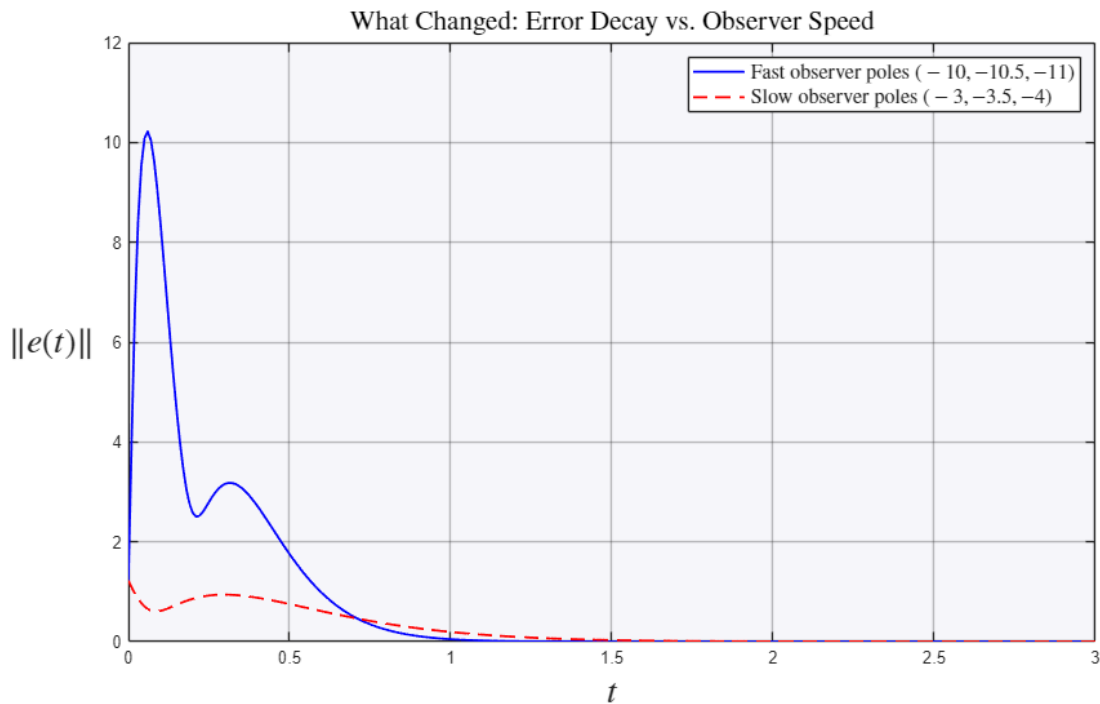
```
observer_poles_slow = [-3,-3.5,-4];
G_slow = acker(A',C',observer_poles_slow)';
fprintf('\n||G|| (fast poles) = %.2f, ||G|| (slow poles) = %.2f\n', ...
        norm(G), norm(G_slow))
```

$\|G\|$ (fast poles) = 442.74, $\|G\|$ (slow poles) = 16.77

What Changes with Observer Pole Speed: Error Convergence

The "before" is a wrong initial estimate (the observer starts at zero while the plant starts away from it); the "after" is the estimate catching up. Re-running with the slower observer poles shows what changes: faster observer poles (larger G) drive the error to zero sooner.

```
A_obs_slow = A - G_slow*C;
sys_obs_slow = ss(A_obs_slow,[B G_slow],eye(3),zeros(3,2));
x_hat_slow = lsim(sys_obs_slow,u_aug',t,x0_hat);
figure
plot(t, vecnorm(x_true-x_hat,2,2),'b','LineWidth',1.3)
hold on
plot(t, vecnorm(x_true-x_hat_slow,2,2),'r--','LineWidth',1.3)
hold off
grid on
legend('Fast observer poles (-10,-10.5,-11)','Slow observer poles (-3,-3.5,-4)', ...
      'Interpreter','latex','FontSize',12)
title('What Changed: Error Decay vs. Observer Speed','Interpreter','latex','FontSize',16)
ylabel('\|e(t)\|','Interpreter','latex','FontSize',20)
set(get(gca,'YLabel'),'Rotation',0)
xlabel('t$','$','Interpreter','latex','FontSize',20)
```



Try it yourself

- Slow the observer poles toward the controller poles and notice the estimate take much longer to catch the true state.
- Start the observer closer to the truth ($x0_hat = [0.8; 0.4; -0.4]$) and see the error start small and vanish quickly.